

GUI 使用指南

项目提供 4 个独立的 PySide6 桌面应用，通过 pybind11 直接调用 C++ 静态库，所有数据通过内存 numpy 数组传递。

四个 GUI 的共享组件（任务栏图标、主题管理、控制台、配置搜索、依赖、对比表等）详见 [SHARED_COMPONENTS.md](#)。

通用构建指南

详细的跨平台编译运行手册见 [BUILD_GUIDE.md](#)。

Windows 一键构建 (推荐)

```
cd NN_GUI_XXX
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
pip install -r requirements-dev.txt
scripts\build.bat
```

输出为 `dist\雷达XX系统.exe`（单文件模式，~90MB），可直接拷贝到目标机器运行。

macOS 一键构建

```
cd NN_GUI_XXX
bash scripts/build.sh setup      # 安装依赖
bash scripts/build.sh run        # 开发运行
bash scripts/build.sh all        # C++ + 打包 .app
```

部署

- 将 `config.json` 放在 exe 上两级目录
- 或通过菜单 **文件** → **加载配置...** 手动导入
- 或设置环境变量 `SPS_CONFIG` 指向 `config.json`

01_GUI_waveform — 波形生成

封装模块 01 的 5 种波形模式。

目录结构

```
01_GUI_waveform/
├─ app.py                      # 程序入口
├─ requirements.txt / requirements-dev.txt
├─ lib/                        # 构建产物 (.gitignore)
│  └─ waveform_cpp.pyd         # C++ 绑定 (windows)
│  └─ waveform_cpp.cpython-*.so # C++ 绑定 (macOS)
├─ scripts/                   # 构建/打包脚本
│  └─ build.sh / build.bat
│  └─ setup_cython.py
│  └─ 雷达波形生成系统.spec / 雷达波形生成系统_win.spec
```

```
|— ui/                                # PySide6 界面
|   |— main_window.py, param_panel.py, plot_panel.py
|   |— console_panel.py, scientific_spinbox.py, theme.py
|— assets/                            # 图标资源
|— core/                              # config_manager.py, signal_utils.py
|— venv/ / dist/
```

支持的波形模式

Case	波形类型	关键参数
1	固定跳频	跳频点数N, 频率步进delta_f
2	随机相位	无额外参数
3	PRI抖动	标称PRT, 抖动范围jitter_us
4	混合(跳频+抖动)	载频分组数fnum, PRT, 抖动
5	复合(跳频+随机相位)	跳频点数N, 频率步进delta_f

可视化 (7 种图表)

时域 | 频谱 | 信号矩阵 | 相位谱 | 载频序列 | 随机相位 | STFT | 距离-多普勒

分步构建 (可选)

C++ 源码已从仓库移除，需先解压加密 zip（密码见 BUILD_GUIDE.md）。

```
REM 第 1 步: 构建静态库
cd 01_waveform_generation && mkdir build && cd build
cmake .. -G "MinGW Makefiles" && cmake --build . --config Release --target waveform_core

REM 第 2 步: 编译 C++ 绑定
cd 01_GUI_waveform && venv\Scripts\activate
g++ -O3 -std=c++17 -shared -I"<pybind11>" -I"<Python>" -I"..\\third_party\\eigen" -I"..\\01_waveform_generation\\include"
..\\01_waveform_generation\\bindings\\waveform_bind.cpp
..\\01_waveform_generation\\build\\libwaveform_core.a -o lib\\waveform_cpp.pyd

REM 第 3 步: 打包
pyinstaller --clean --noconfirm scripts\\雷达波形生成系统_win.spec
```

02_GUI_jamming — 干扰生成

封装模块 02 的 10 种干扰模式。

目录结构

```
02_GUI_jamming/
├─ app.py, requirements.txt, requirements-dev.txt
├─ lib/                                # jamming_cpp.pyd + MinGW DLLs
├─ scripts/                           # build.sh/bat + spec + setup_cython.py
├─ ui/                                # main_window, param_panel, plot_panel, ...
├─ assets/, core/, venv/, dist/
```

支持的干扰模式

Case	干扰类型	缩写	关键参数
1	距离假目标	RDJ	延迟脉冲数jj, 假目标距离Rj, 幅度增益amp_j
2	速度假目标	VDJ	假目标速度Vj, 延迟脉冲数jj, 幅度增益amp_j
3	间歇采样转发	ISRJ	采样周期Ts_ISRJ, 采样脉宽T_ISRJ, 幅度增益amp_j
4	窄带噪声	NNJ	噪声功率power_dBW, 滤波器阶数, 截止频率
5	距离波门拖引	RGPO	拖引速度Vj, 干扰幅度, 拖引阶段数drag_stages
6	速度波门拖引	VGPO	拖引速度Vj, 干扰幅度, 拖引阶段数drag_stages
7	密集复制转发假目标	DRFTJ	干信比JSR, 转发次数num_jam, 距离增量detaR
8	脉内前沿切片重复	IPLESRJ	功率增益A_RJ, 初始距离R0_new, 切片比T_ISRJ_ratio
9	频谱弥散	SMSP	切片数num_slices, 干信比JSR, 额外系数amp_extra
10	梳状谱	COMB	谱线数num_tones, 干信比JSR, 频率间隔deltaf

可视化 (7 种图表)

时域波形 | 频域频谱 | 信号矩阵 | 距离-多普勒 | STFT 时频 | 脉冲对比 | 拖引轨迹 (仅 Case 5/6)

分步构建 (可选)

```
REM 构建静态库
cd 02_jamming_generation && mkdir build && cd build
cmake .. -G "MinGW Makefiles" && cmake --build . --config Release --target jamming_core

REM 编译绑定
cd 02_GUI_jamming && venv\Scripts\activate
g++ -O3 -std=c++17 -shared -I"<pybind11>" -I"<Python>" -I"..\"third_party\"eigen"
-I"..\"third_party\"fftw-install\include" -I"..\"02_jamming_generation\"include"
..\"02_jamming_generation\"bindings\"jamming_bind.cpp
..\"02_jamming_generation\"build\"libjamming_core.a -L"..\"third_party\"fftw-
install\"lib" -lfftw3 -o lib\"jamming_cpp.pyd

REM 打包
pyinstaller --clean --noconfirm scripts\"雷达干扰生成系统_win.spec"
```

03_GUI_detection — 干扰识别与抑制

封装模块 03 的 5 种干扰类型识别。**拥有独立参数体系** (fc=35GHz Ka 波段)。

目录结构

```
03_GUI_detection/
├─ app.py, requirements.txt, requirements-dev.txt
├─ lib/                               # detection_cpp.pyd + MinGW DLLs
├─ scripts/                           # build.sh/bat + spec + setup_cython.py
├─ ui/                                # main_window, param_panel, plot_panel, ...
├─ assets/, core/, venv/, dist/
```

支持的干扰类型

类型	缩写	说明
间歇采样直接转发	ISDJ	周期采样 + 直接转发
间歇采样重复转发	ISRJ	周期采样 + 重复转发重组
间歇采样循环转发	ISCJ	周期采样 + 循环转发
窄带瞄频噪声	NBJ	复高斯白噪声 + 载波调制 + 低通滤波
距离欺骗干扰	RDJ	假目标距离延迟

参数面板

分为 5 个区域：系统参数 (fc=35GHz/B=80MHz/R0=1000m 等 13 项)、识别参数 (gr_stft_num, gr_hamming_len)、分离参数 (divi_stft_num, divi_hamming_len, tsallis_q=1.2)、时频参数 (scale_factor)、生成器参数 (iscj_sub_T, nbj_center_freq 等 6 项)。

可视化 (8 种图表)

标签	说明
时域	单脉冲波形 (含干扰回波/含噪目标/分离干扰/分离目标)
频域	FFT 幅度谱 (dB)
STFT	C++ 计算的 STFT 时频图, Jet 色表
干扰定位	STFT 幅度 + Otsu 二值掩模 (红色半透明)
分离对比	三线叠加对比 (含干扰回波/分离干扰/分离目标)
分离时频	scipy.signal.stft 分离后时频图
距离-多普勒	RD 热力图 (物理坐标 m/m/s)
检测统计	柱状图 (4种识别投票) + JSR 抑制比

CPI 选择器: 脉冲索引下拉框, 实时切换时域/频域/分离视图。

RD 图坐标映射:

```
carrier_removal = exp(-j * 2π * (fc/fs) * n)
ref = exp(j * π * gama * t²)
xi = fftshift(fftshift(fftshift(nrn, 1/fs)) * c / (2*gama)
dv = fftshift(fftshift(fftshift(cpiNum, 1/prf)) * lambda / 2
```

返回数据结构

字段	类型	说明
echo_signal	complex128 (cpiNum×nrn)	含干扰回波 (全部 CPI)
stft_matrix	complex128 (STFT_NUM×nrn)	STFT 时频矩阵
jam_mask	float64 (STFT_NUM×nrn)	Otsu 二值掩模
detection_types	int64 (cpiNum,)	每个CPI识别结果
jamming_signal	complex128 (nrn,)	分离干扰信号
target_signal	complex128 (nrn,)	分离目标信号
dominant_type	int	主导干扰类型
jsr	float	干扰抑制比 (dB)
elapsed	float	计算耗时 (s)
log_output	str	C++ 后端日志

C++ 模块关系

```
03_GUI_detection → detection_cpp.pyd → libdetection_core.a
→ EchoGenerator.h, JammingSimulator.h, GrDetection.h
→ JamTarDivi.h (q=1.2), TfrStft.h, JamLocated.h
```

04_GUI_signal_processing — 信号处理

封装模块 04 + 模块 01，是唯一链接两个静态库的 GUI。支持 6 种处理模式。

目录结构

```
04_GUI_signal_processing/
├─ app.py, requirements.txt, requirements-dev.txt
├─ lib/                                # signal_processing_cpp.pyd + MinGW DLLs
├─ scripts/                            # build.sh/bat + spec + setup_cython.py
├─ ui/                                 # main_window, param_panel, plot_panel, ...
├─ assets/, core/, tests/, venv/, dist/
```

支持的处理模式

Case	处理类型	数据来源
1	跳频信号处理	模块 01 跳频波形
2	固定载频处理	模块 01 固定载频波形
3	传统脉冲压缩	模块 01 LFM 波形
4	改进型脉冲压缩	模块 01 NLFM 波形
5	复合处理	模块 01 复合波形
6	干扰解耦	模块 02 含干扰回波 (5种干扰类型)

参数面板

- 分为 3 个区域：
- **系统参数** (fc=16GHz/B=40MHz/Vr=50m/Rs=10000m 等 10 项)
 - **波形参数** (Case1~5 各有独立面板: 跳频点数、抖动范围等)
 - **解耦参数** (Case6: divi_stft_num=256, tsallis_q=2.0, SNR=25dB, JSR=30dB 等)

可视化 (7 种图表)

标签	说明
脉冲特性	每脉冲峰值频率 + 跨脉冲解卷绕相位
频谱	单脉冲 FFT 频谱 (dB)
信号矩阵	热力图 (实部/虚部/幅度/相位)
距离-多普勒	RD 热力图 (C++ 返回物理坐标 xi/dv)
解耦结果	Case6 分离对比 (三线叠加)
结果汇总	Case1~5 峰值功率柱状图 / Case6 抑制比柱状图
STFT	scipy.signal.stft 时频分析 (去载波后)

RD 图坐标 (C++ 直接返回):

```
xi = fftshift(fftshift(fftshift(nrn, 1/fs)) * c / (2*gama) # 距离 (m)
dv = fftshift(fftshift(fftshift(nan1, 1/prf)) * lambda / 2 # 速度 (m/s)
```

STFT 载波去除:

```
carrier_phase = 2π * (fc/fs) * n
col_bb = col * exp(-1j * carrier_phase)
```

C++ 入口函数

- run_processing_rd(case_num, config_dict) — Case1~5:
- 输出: rd_map (nrn×nan1), xi, dv, input_signal, nrn, nan1, elapsed, log_output
- run_processing_decouple(jam_type, config_dict) — Case6:
- 输出: jam_signal, target_signal, input_signal, decouple_flag, jsr_dB, avg_threshold, gaojiepu_count, nrn, nan1, elapsed, log_output

C++ 模块关系

```
04_GUI_signal_processing → signal_processing_cpp.pyd
├→ libsignal_processing_core.a (Module3.h: chuli_Case1~5)
└→ libwaveform_core.a (waveform_core.h: generate_waveform)
```

Case6 解耦: JamTarDivi (q=1.0) + EchoGenerator4 + JammingSimulator4

Case6 参数体系: 使用 `detection_suppression.*` 路径 (与模块 03 相同), 但 Tsallis q=1.0 (模块 03 为 1.2)。